LP-based branch-and-bound (1/2)

Outline

Why a branch-and-bound algorithm?

The idea in words and in a picture

Definitions and notation

The algorithm

A short recap of the dual simplex algorithm

Branching-variable selection strategies

Branching-node selection strategies

Learning objectives

At the end of this chapter you will be able to:

- explain how branching operations divide an ILP problem into smaller LP problems, and why no feasible solution is lost;
- apply the three pruning criteria, and decide when the bound of a node can be rounded down;
- execute an LP-based branch-and-bound algorithm by hand, re-optimizing every node with the dual simplex algorithm;
- compute the lower bound, the upper bound and the optimality gap at every iteration;
- compare different branching-variable and branching-node selection strategies on the same instance.

Prerequisites

- From the module *Linear Programming*, Chapter 10 «Dual Simplex algorithm»: simplex dictionaries, dual feasible bases and dual pivoting operations (a short recap is given in this chapter)
- From this module, the chapter «Integer linear programming»: ILP problems, feasible and optimal solutions, and the car example
- From this module, the chapter «Primal and dual bounds»: the LP relaxation, lower bounds from feasible solutions, upper bounds from the LP relaxation, and the optimality gap

Why a branch-and-bound algorithm?

Introduction: Why a branch-and-bound algorithm?

- We start again from the car example, the guiding example of the module
- Its LP relaxation gives a fractional solution and bounds on $z(\mathrm{ILP})$, but not an optimal solution
- Listing all the integer points works here, but not on larger problems
- We need a method that **finds** an optimal solution and **proves** its optimality solving only a few LP problems

The car example

- We start from the **car example**: x_1 and x_2 are the numbers of cars of two types
- The ILP problem reads as follows:

$$
\begin{array}{rrcrcr}
\max & 3\,x_1 & + & 2\,x_2 & & \\
\text{subject to} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\
 & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\
 & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0}
\end{array}
$$

The LP relaxation of the car example

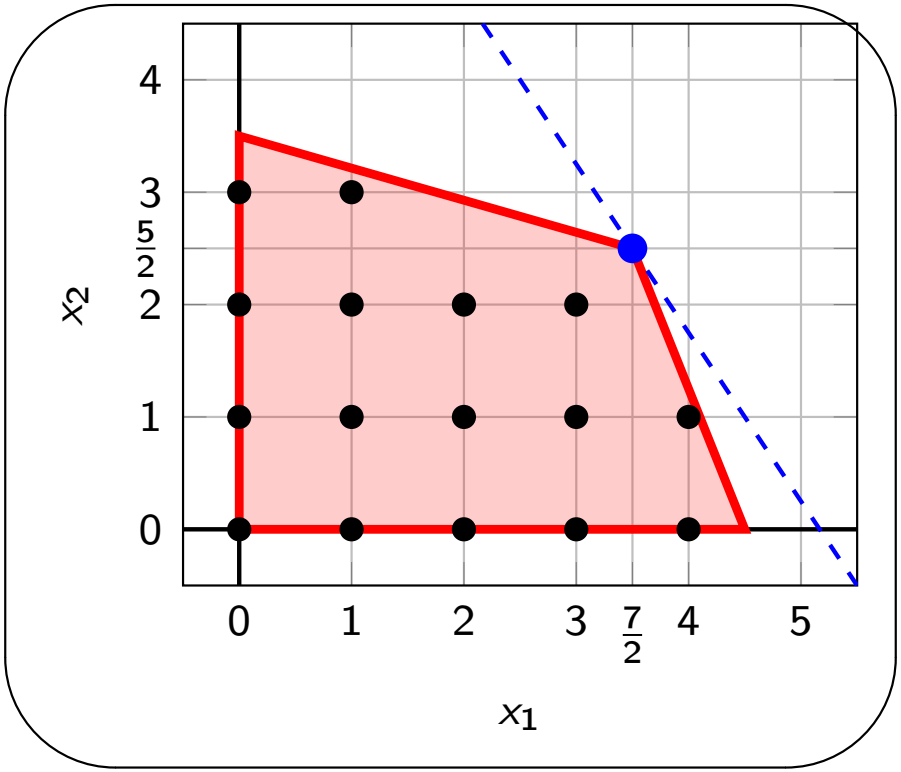
- The optimal solution of the LP relaxation is fractional:

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2}, \qquad z(\mathrm{LP}) = \frac{31}{2}.$$

- Rounding it down we get the feasible solution $\bar{\boldsymbol{x}} = (3, 2)$ of value 13
- Since the costs are integer, the LP relaxation also gives the upper bound $\lfloor z(\mathrm{LP}) \rfloor = 15$:

$$13 \quad \leq \quad z(\mathrm{ILP}) \quad \leq \quad 15.$$

The integer points of the car example



The feasible region of the LP relaxation (red), its 16 integer points (black) and the optimal solution of the LP relaxation (blue), with the objective line through it (dashed).

Enumeration does not scale

- The feasible region contains 16 integer points
- Enumerating them all is still possible here, but the number of integer points grows exponentially with the number of variables
- We need a way to exclude whole groups of integer points without visiting them

A challenge

Before reading on, try to answer two questions.

Which feasible solution is **optimal**?

How can we prove that no better solution exists,
solving only a few LP problems
and without listing all the integer points?

- The worked examples of this chapter answer both questions

Summary

- The LP relaxation of the car example has the fractional optimal solution $(\frac{7}{2}, \frac{5}{2})$, with $z(\mathrm{LP}) = \frac{31}{2}$
- Rounding and the LP relaxation give $13 \leq z(\mathrm{ILP}) \leq 15$, but no certificate of optimality
- Complete enumeration of the integer points does not scale
- The branch-and-bound algorithm closes the gap solving a sequence of LP problems

The idea in words and in a picture

Introduction: The idea in words and in a picture

- Before the formal definitions, we see the idea of the algorithm on the car example
- Two operations: **branching** divides the problem, **bounding** decides which parts must still be explored
- A single picture shows both of them

LP-based branch-and-bound

- The **LP-based branch-and-bound algorithm** (**B&B algorithm**, for short) is an exact method for ILP problems
- It is called **LP-based** because it solves LP relaxations to guide the search for an optimal solution

Branching operations

- In the optimal solution of the LP relaxation of the car example the variable x_1 takes a fractional value between 3 and 4
- Every feasible solution of the ILP problem satisfies either $x_1 \leq 3$ or $x_1 \geq 4$
- So we can split the problem into two smaller LP problems, one for each side, and remove the strip in between, which contains no integer point:

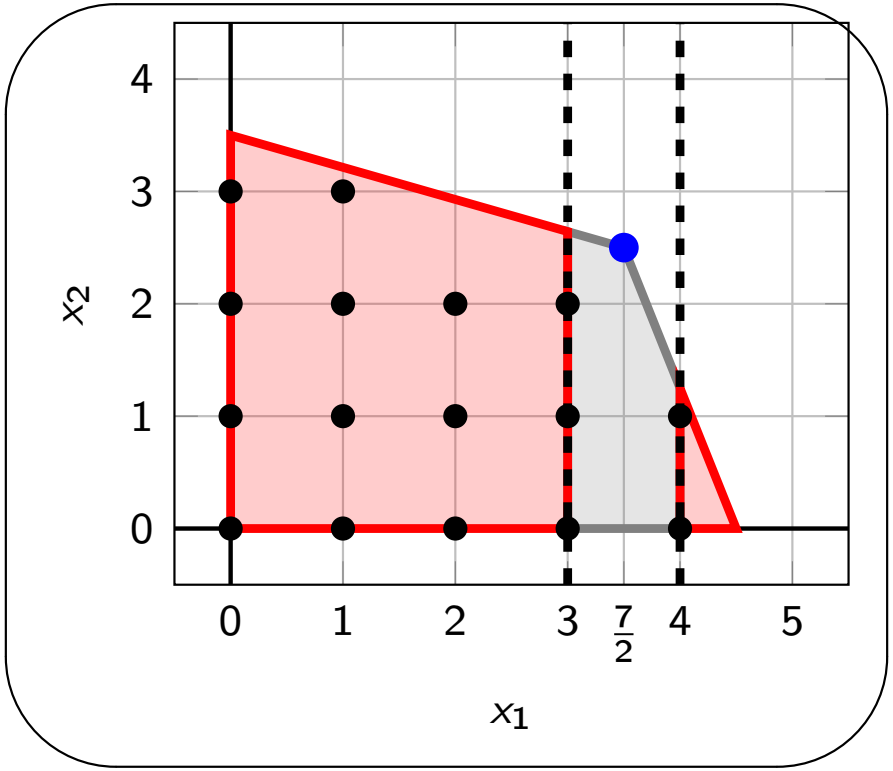
$$\rightsquigarrow \textbf{branching operations}.$$

Bounding operations

- Each new LP problem is solved
- Its optimal value bounds from above the value of every solution in its region
- So it tells us whether the region must be explored further or can be discarded:

$$\rightsquigarrow \textbf{bounding operations}.$$

Branching on x_1 at the root



The two red regions are the feasible regions of the children ($x_1 \leq 3$ and $x_1 \geq 4$); the gray strip $3 < x_1 < 4$ is cut off, and it contains no integer point.

Implicit enumeration

A branch-and-bound algorithm explores implicitly
all the feasible solutions:

it divides the feasible region into smaller regions
(**branching**)

and discards every region that cannot contain
a better solution than the best one found so far
(**bounding**).

Summary

- The B&B algorithm is an exact method for ILP problems, guided by LP relaxations
- **Branching**: a fractional value $3 < \tilde{x}_1 < 4$ splits the problem into $x_1 \leq 3$ and $x_1 \geq 4$; the strip in between contains no integer point
- **Bounding**: the optimal value of each new LP problem tells whether its region must be explored further
- All the feasible solutions are explored **implicitly**

Definitions and notation

Introduction: Definitions and notation

- We fix the problem: an ILP problem in canonical maximization form with integer data
- We define the objects of the algorithm: the branch-and-bound tree, the incumbent solution, the branching and bounding operations, the list of LP problems
- We define the lower bound, the upper bound and the optimality gap that measure the progress of the search

ILP problems in canonical maximization form

- Given n values $c_j \in \mathbb{Z}$, $m \cdot n$ values $a_{ij} \in \mathbb{Z}$ and m values $b_i \in \mathbb{Z}$, an **integer linear programming** (ILP) problem in canonical maximization form, with n integer variables and integer data, reads as follows:

$$\max \quad \sum_{j=1}^{n} c_j \, x_j$$

$$\text{subject to} \quad \sum_{j=1}^{n} a_{ij} \, x_j \leq b_i, \qquad \forall i \in \{1, 2, \ldots, m\},$$

$$x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \ldots, n\}.$$

Notation

- The car example is an ILP problem with $n = 2$ and $m = 2$
- We denote by $z(\mathrm{ILP})$ the optimal value of the ILP problem, and by $\tilde{\boldsymbol{x}}(\mathrm{ILP})$ an optimal solution
- The **LP relaxation** of the ILP problem, called the LP_1 problem, is obtained by replacing the integrality constraints $x_j \in \mathbb{Z}_{\geq 0}$ with the non-negativity constraints $x_j \geq 0$, for all $j \in \{1, 2, \ldots, n\}$

The branch-and-bound tree

The **branching operations** generate
a **branch-and-bound tree** containing LP problems.

Each LP problem is obtained from the LP relaxation
by adding specific **branching constraints**.

- The recursive structure of a B&B algorithm induces the tree
- Each **branching node** of the tree corresponds to an LP problem; the root is the LP_1 problem

Definition 1: branching node index

For each branching node k, the associated LP problem is denoted by LP_k. If LP_k has an optimal solution, it is denoted by $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ and its optimal value by $z(\mathrm{LP}_k)$.

*The integer k is called the **branching node index**: it identifies the node uniquely, and the nodes are numbered in the order in which their LP problems are solved.*

Incumbent solution and lower bound

- A B&B algorithm keeps the best feasible solution of the ILP problem found during the search
- This solution is called the **incumbent solution** and is denoted by $\bar{\boldsymbol{x}}$; its value is a **lower bound** LB on $z(\mathrm{ILP})$
- Before any feasible solution is found we set $\mathrm{LB} = -\infty$
- The lower bound and the incumbent solution are updated whenever a better feasible solution is found. At termination we have an **algorithmic certificate** that

$$z(\mathrm{ILP}) = \mathrm{LB} \qquad \text{and} \qquad \tilde{\boldsymbol{x}}(\mathrm{ILP}) = \bar{\boldsymbol{x}}.$$

Binary branching

- The most common branching operation is the **binary branching**
- It acts on a single **branching variable** and produces two child branching nodes:
  - one with an additional **upper bound** on the branching variable
  - one with an additional **lower bound** on the branching variable

Down-branching and up-branching constraints

- Let $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ be an optimal solution of the LP_k problem, and let x_j, with $j \in \{1, 2, \ldots, p\}$, be an integer variable that takes a fractional value $\tilde{x}_j$ (the integer variables are $x_1, \ldots, x_p$; in an ILP problem $p = n$)
- The binary branching on x_j generates two child LP problems:

1. the LP_k problem with the additional **down-branching constraint**
$$x_j \leq \lfloor \tilde{x}_j \rfloor ;$$
2. the LP_k problem with the additional **up-branching constraint**
$$x_j \geq \lfloor \tilde{x}_j \rfloor + 1.$$

Correctness of the branching

- The two child nodes are called the **down-branching node** and the **up-branching node**
- In both of them $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ is no longer feasible

Every feasible solution of the ILP problem
contained in the region of LP_k satisfies either

$$x_j \leq \lfloor \tilde{x}_j \rfloor \quad \text{or} \quad x_j \geq \lfloor \tilde{x}_j \rfloor + 1,$$

so it is contained in the region of one of the two children.

This ensures the correctness of the B&B algorithm.

Bounding operations

- The value $z(\mathrm{LP}_k)$ is an upper bound on the value of every feasible solution of the ILP problem in the region of LP_k
- When all the values are integer, this bound can be rounded down

Observation 1: rounding the bound of a node

Since all the variables and all the costs are integer, every feasible solution in the region of LP_k has value at most $\lfloor z(\mathrm{LP}_k) \rfloor$.

Proof

- Since $c_j \in \mathbb{Z}$ for all j, the value

$$\sum_{j=1}^{n} c_j \bar{x}_j$$

of a feasible solution $\bar{\boldsymbol{x}}$ is a sum of products of integers, hence an integer

- Since it is at most $z(\mathrm{LP}_k)$, it is at most the largest integer not exceeding $z(\mathrm{LP}_k)$, that is, $\lfloor z(\mathrm{LP}_k) \rfloor$

□

The bound of a node

- Accordingly, we call **bound of the node** k the value

$$\mathrm{UB}(\mathrm{LP}_k) = \begin{cases} \lfloor z(\mathrm{LP}_k) \rfloor & \text{if } p = n \text{ and } c_j \in \mathbb{Z} \\ & \text{for all } j \in \{1, 2, \ldots, n\}, \\ z(\mathrm{LP}_k) & \text{otherwise.} \end{cases}$$

- The bound is rounded down only when all the variables are integer ($p = n$) and all the costs are integer

Pruning criteria (1/2)

- Given a branching node k, no branching operation is performed on it if one of the following **pruning criteria** applies. In these cases the node is said to be **pruned**

1. **Pruning by infeasibility**: the LP_k problem is infeasible, so its region contains no feasible solution of the ILP problem
2. **Pruning by integrality**: the integer variables have integer values in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$, so $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ is feasible for the ILP problem, and it is optimal in the region of LP_k. If $z(\mathrm{LP}_k) > \mathrm{LB}$, the lower bound and the incumbent solution are updated

Pruning criteria (2/2)

3. **Pruning by bound**: the bound of the node does not exceed the lower bound,

$$\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB},$$

so the region of LP_k contains no solution better than the incumbent one

The criteria are checked in this order:
infeasibility, integrality, bound.

When both integrality and bound apply,
the node is pruned by integrality.

The list of LP problems

- A branching node that is not pruned is inserted into a **list** L of LP problems waiting for a branching operation
- The order in which the LP problems are extracted from the list determines the **branching-node selection strategy**
- The choice of the branching variable determines the **branching-variable selection strategy**
- Both are discussed after the algorithm

Upper bound

- If the list L is not empty, every feasible solution better than the incumbent one lies in the region of some node of the list
- Accordingly, the B&B algorithm has the **upper bound**

$$z(\mathrm{ILP}) \quad \leq \quad \mathrm{UB} = \max_{k \in L} \{\mathrm{UB}(\mathrm{LP}_k)\}\,.$$

- When the list is empty, the incumbent solution is optimal and $\mathrm{UB} = \mathrm{LB}$

Optimality gap

- The upper bound is computed after the solution of the root and updated at the end of every iteration
- The algorithm does not need it, but it measures the progress through the **optimality gap**:

$$\text{optimality gap} = \frac{\text{UB} - \text{LB}}{|\text{LB}|}, \qquad \text{for } \text{LB} \neq 0.$$

Heuristic initialization

Before the B&B algorithm, a **heuristic algorithm** can be run to find a good feasible solution.

The lower bound LB and the incumbent solution $\bar{x}$ are then initialized with this solution.

- A good initial lower bound allows the algorithm to prune nodes by bound earlier, and it can reduce the size of the branch-and-bound tree considerably
- If the initial lower bound already satisfies $\mathrm{UB}(\mathrm{LP}_1) \leq \mathrm{LB}$, the root is pruned by bound: the heuristic solution is optimal and no branching is needed

Summary (1/2)

Problem and tree:

- ILP problem in canonical maximization form with integer data; the root is the LP relaxation LP_1
- Each node k is an LP problem LP_k; the nodes are numbered in the order in which they are solved

Branching:

- On a fractional integer variable: $x_j \leq \lfloor \tilde{x}_j \rfloor$ and $x_j \geq \lfloor \tilde{x}_j \rfloor + 1$
- No feasible solution of the ILP problem is lost

Summary (2/2)

Bounding:

- Bound of the node: $\mathrm{UB}(\mathrm{LP}_k) = \lfloor z(\mathrm{LP}_k) \rfloor$ when all the variables and costs are integer
- Pruning by infeasibility, by integrality, by bound ($\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB}$), in this order

Bounds of the search:

- LB from the incumbent solution, $\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k)$ from the list
- Optimality gap $(\mathrm{UB} - \mathrm{LB})/|\mathrm{LB}|$; a heuristic solution can initialize LB

The algorithm

Introduction: The algorithm

- We put the operations together in the pseudocode of the B&B algorithm and of its sub-procedure
- We see the skip rule, which avoids solving useless children, and how the nodes are numbered
- We count the nodes of a complete tree: this explains why pruning is essential

The B&B algorithm (1/3)

Input: An ILP problem in canonical maximization form with integer input data.

Output: An optimal solution to the ILP problem (if one exists) or a certificate of infeasibility/unboundedness.

Step 0: **Initialization**

0.1 Relax the integrality constraints and solve the LP_1 problem.

1. if $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$ is **integer**, then it is an optimal solution and $z(\mathrm{LP}_1)$ is the optimal value of the ILP problem. `STOP`.
2. if the LP_1 problem is **infeasible**, then the ILP problem is infeasible. `STOP`.
3. if the LP_1 problem is **unbounded**, then the ILP problem is either infeasible or unbounded. `STOP`.

The B&B algorithm (2/3)

Step 0: Initialization (continued)

0.2 Insert the LP_1 problem into a **list of LP problems for branching** (denote L). Set the **lower bound** $\mathrm{LB} \leftarrow -\infty$ (or the value of a heuristic solution), and set a branching node counter $c \leftarrow 1$.

Step 1: **Optimality check**

0.1 if the list L is empty and $\mathrm{LB} \neq -\infty$, then LB is the optimal value and the **incumbent solution** $\bar{\boldsymbol{x}}(\mathrm{ILP})$ is an optimal solution to the ILP problem. `STOP`.

0.2 if the list L is empty and $\mathrm{LB} = -\infty$, then the ILP problem is infeasible. `STOP`.

Step 2: **Branching-node selection strategy**

Select and remove an LP_k problem from the list L.

The B&B algorithm (3/3)

Step 3: **Branching-variable selection strategy**

Select a variable x_j with fractional value $\tilde{x}_j$ in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ and generate two new LP problems:

0.1 the LP_{c+1} problem $\rightsquigarrow$ the LP_k problem with the additional **down-branching constraint**: $x_j \leq \lfloor \tilde{x}_j \rfloor$

0.2 the LP_{c+2} problem $\rightsquigarrow$ the LP_k problem with the additional **up-branching constraint**: $x_j \geq \lfloor \tilde{x}_j \rfloor + 1$

Execute the **Branching Node Solution Sub-Procedure** for the LP_{c+1} problem and for the LP_{c+2} problem; the second one is skipped if, after the first, $\lfloor z(\mathrm{LP}_k) \rfloor \leq \mathrm{LB}$. Increment the branching node counter $c \leftarrow c + 2$ and go to step 1.

Each child is solved and classified
by the **Branching Node Solution Sub-Procedure**:
this is its pseudocode.

Branching Node Solution Sub-Procedure

Input: An LP_k problem associated with a branching node.

Step 1: **Bounding operations**

Solve the LP_k problem.

0.1 if the LP_k problem is infeasible, then `STOP`.

0.2 if $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ is integer, then, if $z(\mathrm{LP}_k) > \mathrm{LB}$, update the **lower bound** and the **incumbent solution**:

$$\mathrm{LB} \leftarrow z(\mathrm{LP}_k) \quad \text{and} \quad \bar{\boldsymbol{x}}(\mathrm{ILP}) \leftarrow \tilde{\boldsymbol{x}}(\mathrm{LP}_k)$$

Remove from the list L all LP problems with $\lfloor z(\mathrm{LP}) \rfloor \leq \mathrm{LB}$ and `STOP` (also when $z(\mathrm{LP}_k) \leq \mathrm{LB}$).

0.3 if $\lfloor z(\mathrm{LP}_k) \rfloor \leq \mathrm{LB}$, then `STOP`.

Step 2: **Insertion into the list**

Insert the LP_k problem into the list L and `STOP`.

Remarks on the pseudocode

- The LP problem with the down-branching constraint is called the ***down-branching node***; the LP problem with the up-branching constraint is called the ***up-branching node***
- The order in which they are solved determines the order in which the LP problems enter the list L
- The pseudocode does not say how the LP problems are solved: any method for LP problems can be used
- If the **dual simplex algorithm** is used, each child is solved starting from the optimal dictionary of its parent: after adding the branching constraint this dictionary is still dual feasible, so in general a few pivot operations are enough (see the recap of the dual simplex algorithm)

The skip rule

- The bound of a child never exceeds the bound of its parent, since the child has one more constraint
- Accordingly, if after the solution of the first child we have $\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB}$, the second child cannot contain a better solution, and it is **not solved**
- This is the **skip rule**

The numbering of the nodes

- A skipped child does not receive a node index
- The nodes are numbered in the order in which their LP problems are solved, so the indices of the solved nodes are always $1, 2, \ldots, c$
- The order in which the two children are solved also determines the order in which they enter the list L
- In these slides the down-branching node is solved first, unless otherwise stated

Number of branching nodes

- A **complete** binary branch-and-bound tree with $n \geq 1$ levels has 2^{n-1} leaf branching nodes
- Along every root-to-leaf path exactly $n-1$ branching constraints are imposed

Observation 2

The total number of branching nodes in a complete binary branch-and-bound tree with $n \geq 1$ levels is:

$$\sum_{j=0}^{n-1} 2^j = 2^n - 1.$$

Proof (1/2)

- By induction on n
- **Base case.** For $n = 1$ the statement reads

$$\sum_{j=0}^{0} 2^j = 2^1 - 1,$$

that is, $1 = 1$, which is true

Proof (2/2)

- **Inductive step.** Suppose that the statement holds for n; we prove it for $n+1$. We have:

$$\begin{aligned}\sum_{j=0}^{n} 2^j &= \sum_{j=0}^{n-1} 2^j + 2^n \\ &= 2^n - 1 + 2^n \quad \text{(by the inductive hypothesis)} \\ &= 2^{n+1} - 1,\end{aligned}$$

which is the statement for $n+1$

□

A geometric progression

- The total number of nodes is the sum of the first n terms of a geometric progression with ratio $r = 2$:

$$\sum_{j=0}^{n-1} r^j = \begin{cases} \dfrac{r^n - 1}{r - 1} & \text{if } r \neq 1, \\ n & \text{otherwise.} \end{cases}$$

- With p binary variables the complete tree has $p + 1$ levels and $2^{p+1} - 1$ nodes: the pruning criteria are what makes the algorithm practical

Summary

- The B&B algorithm extracts a node from the list, branches on a fractional variable and solves the two children with the sub-procedure
- The sub-procedure prunes the child by infeasibility, by bound or by integrality (updating LB and the incumbent solution), or inserts it into the list
- **Skip rule**: the second child is not solved if, after the first, $\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB}$; skipped children receive no index
- A complete binary tree with n levels has $2^n - 1$ nodes: pruning is essential

A short recap of the dual simplex algorithm

Introduction: A short recap of the dual simplex algorithm

- Every node of the tree is an LP problem, and solving each of them from scratch is wasteful
- We see how a branching constraint becomes a new row of the optimal dictionary of the parent
- The new dictionary is dual feasible: the dual simplex algorithm re-optimizes it with a few pivoting operations

Re-optimizing a child

- A child differs from its parent by one constraint
- Solving it from scratch would waste the work done for the parent
- The B&B algorithm starts instead from the **optimal dictionary of the parent** and re-optimizes it with the **dual simplex algorithm**

The row of the branching variable

- Let x_j be the branching variable. It is basic in the optimal dictionary of LP_k, since its value is fractional, and its row reads

$$x_j = \bar{b}_j - \sum_{q \in \mathcal{N}} \bar{a}_{jq} x_q, \qquad \text{with } \bar{b}_j = \tilde{x}_j.$$

- At depth d of the tree, the branching constraint receives the slack (or surplus) variable x_s, with $s = n + m + d$

Observation 3: the row of the branching constraint (1/2)

Substituting the row of x_j, the branching constraints become new basic rows of the dictionary. The down-branching constraint gives:

$$x_j \leq \lfloor \tilde{x}_j \rfloor \quad \Longrightarrow \quad x_s = \lfloor \tilde{x}_j \rfloor - x_j$$

$$= \left(\lfloor \tilde{x}_j \rfloor - \tilde{x}_j \right) + \sum_{q \in \mathcal{N}} \bar{a}_{jq} \, x_q .$$

Observation: the row of the branching constraint (2/2)

The up-branching constraint gives:

$$x_j \geq \lfloor \tilde{x}_j \rfloor + 1 \quad \Longrightarrow \quad x_s = x_j - \lfloor \tilde{x}_j \rfloor - 1$$

$$= \left(\tilde{x}_j - \lfloor \tilde{x}_j \rfloor - 1\right) - \sum_{q \in \mathcal{N}} \bar{a}_{jq}\, x_q .$$

In both cases the value of x_s is negative, while the reduced costs do not change: the new dictionary is ***dual feasible*** *but not primal feasible.*

The starting point of the dual simplex algorithm

A dual feasible dictionary that is not primal feasible
is exactly the starting point
of the **dual simplex algorithm**.

- We use the same pivoting rules as in the module *Linear Programming*, recalled on the next two slides

Dual pivoting operation (1/2)

- A **dual pivoting operation** acts on a dictionary

$$x_i = \bar{b}_i - \sum_{j \in \mathcal{N}} \bar{a}_{ij} x_j, \quad i \in \mathcal{B}, \qquad z = \bar{z} + \sum_{j \in \mathcal{N}} \bar{c}_j x_j,$$

  with $\bar{c}_j \leq 0$ for all $j \in \mathcal{N}$

a. the **pivot row** is the basic variable with negative value having the smallest index:

$$r = \min \left\{ \; i \in \mathcal{B} : \bar{b}_i < 0 \; \right\};$$

b. if $\bar{a}_{rj} \geq 0$ for all $j \in \mathcal{N}$, the LP problem is **infeasible**;

Dual pivoting operation (2/2)

c. otherwise the **pivot column** x_p attains the minimum ratio (ties: smallest index)

$$\frac{\bar{c}_p}{\bar{a}_{rp}} = s_m = \min\left\{ \frac{\bar{c}_j}{\bar{a}_{rj}} : j \in \mathcal{N},\ \bar{a}_{rj} < 0 \right\};$$

d. after the pivoting operation the objective value is $\bar{z} + s_m \bar{b}_r \leq \bar{z}$.

- The algorithm stops when all the basic variables are non-negative: the dictionary is then optimal
- In a child node one or two dual pivoting operations are often enough, while the simplex algorithm from the slack basis would need several pivoting operations for every node

Summary

- Each child starts from the **optimal dictionary of its parent**
- The branching constraint, rewritten with the row of x_j, is a new basic row with a **negative** value; the reduced costs do not change
- The new dictionary is **dual feasible**: the dual simplex algorithm (pivot row of smallest index, minimum ratio) re-optimizes it
- If the pivot row has no negative coefficient, the child is **infeasible**

Branching-variable selection strategies

Introduction: Branching-variable selection strategies

- When several integer variables are fractional, the algorithm must choose the branching variable
- The choice has a large impact on the size of the branch-and-bound tree
- We see four strategies: lexicographic, minimum and maximum infeasibility, strong branching, and we apply them to the root of the car example

Choosing the branching variable

- The choice of the branching variable has a large impact on the size of the branch-and-bound tree

Ideally, the branching variable gives the largest decrease of the bounds of the children and leads quickly to feasible solutions.

However, the time spent to choose it also counts: an expensive rule may slow down the whole algorithm.

- In all the strategies below, ties are broken by the **smallest index**

Lexicographic

The **lexicographic** strategy selects
the integer variable with a fractional value
having the smallest (or the largest) index.

- It requires no computation, but it takes into account neither the decrease of the bounds nor the search for feasible solutions
- It is the rule used in the exercises, because it makes the execution easy to reproduce

Minimum and maximum infeasibility

The **minimum** (respectively, **maximum**) **infeasibility** strategy
selects the variable whose fractional value is the closest to (respectively, the farthest from) an integer value.

- Given a branching node k, for each integer variable x_j with a fractional value $\tilde{x}_j$ in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$, the **fractional value** of the variable is
$$f_j = \min\{\tilde{x}_j - \lfloor \tilde{x}_j \rfloor,\ \lceil \tilde{x}_j \rceil - \tilde{x}_j\}\,.$$

The two infeasibility rules

- **Minimum infeasibility** tries to reduce the number of fractional variables in the children, to reach feasible solutions faster
- **Maximum infeasibility** removes first the variables that contribute more to the infeasibility of $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$
- Neither rule looks at the bounds of the children

Example: fractional values

- At the root node of the car example we have

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2},$$

and the fractional values are:

$$f_1 = \min\left\{\frac{7}{2} - 3,\ 4 - \frac{7}{2}\right\} = \frac{1}{2}$$

$$f_2 = \min\left\{\frac{5}{2} - 2,\ 3 - \frac{5}{2}\right\} = \frac{1}{2}$$

- The two values are equal: both minimum and maximum infeasibility have a tie, which is broken by the smallest index, so both strategies select x_1

Strong branching

The **strong branching** strategy selects the variable that gives the largest decrease of the bounds of the children.

- Given a branching node k, for each integer variable x_j with a fractional value, we solve the two children and compute the decreases of the optimal value

$$\delta_k^d(x_j) = z(\mathrm{LP}_k) - z(\text{down-branching child}),$$

$$\delta_k^u(x_j) = z(\mathrm{LP}_k) - z(\text{up-branching child}),$$

  where the decrease is $+\infty$ for an infeasible child

The strong-branching variable

- The branching variable is x_s, with

$$s = \arg\max \left\{ \min \left\{ \delta_k^d(x_j), \delta_k^u(x_j) \right\} : \right.$$
$$\left. j \in \{1, 2, \ldots, p\},\ \tilde{x}_j \text{ fractional} \right\}.$$

- Both children of the selected variable have a large decrease, so both are more likely to be pruned by bound

- Strong branching is expensive, since it solves two LP problems for every candidate variable

- For this reason it is typically applied only at the top levels of the tree

Example: strong branching (1/2)

- At the root node of the car example we have

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2} \qquad \text{and} \qquad z(\mathrm{LP}_1) = \frac{31}{2}$$

- For each fractional variable we solve the two child LP problems (with the dual simplex algorithm, from the optimal dictionary of LP_1)

- The results are in the table on the next slide

Example: strong branching (2/2)

Variable	Down	z	$\delta_1^d(x_j)$	Up	z	$\delta_1^u(x_j)$	$\min\{\delta_1^d, \delta_1^u\}$
x_1	$x_1 \leq 3$	$\frac{100}{7}$	$\frac{17}{14}$	$x_1 \geq 4$	$\frac{29}{2}$	1	1
x_2	$x_2 \leq 2$	$\frac{151}{10}$	$\frac{2}{5}$	$x_2 \geq 3$	$\frac{45}{4}$	$\frac{17}{4}$	$\frac{2}{5}$

- The largest minimum decrease is 1: strong branching selects x_1, the same variable chosen by the smallest-index rule

Further strategies

- Solvers estimate the decreases δ_k^d and δ_k^u from the history of the search, without solving the children, by means of **pseudocosts**
- The same estimates give a branching-node selection rule called **best estimate**
- These strategies are a possible extension of this chapter

Summary

- **Lexicographic**: smallest (or largest) index; no computation, easy to reproduce
- **Minimum / maximum infeasibility**: fractional value f_j closest to / farthest from an integer
- **Strong branching**: largest $\min\{\delta_k^d, \delta_k^u\}$; expensive, used at the top levels
- On the root of the car example all of them select x_1 (ties broken by the smallest index)

Branching-node selection strategies

Introduction: Branching-node selection strategies

- The order in which the nodes of the list L are processed is the **branching-node selection strategy**: it decides where the search goes next
- We see three strategies: best bound, LIFO and FIFO
- On a complete tree we compare the order in which LIFO and FIFO visit the nodes

Best bound

The **best-bound** strategy selects the node of the list L with the largest optimal value $z(\mathrm{LP}_k)$.

Ties are broken by the smallest node index.

- The selected node gives the upper bound $\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k)$: processing it is the only way to decrease UB
- The rule tends to explore few nodes, but the list can become long

LIFO

The **Last-In-First-Out** (LIFO) strategy processes the nodes in the reverse order of insertion into the list L.

It is equivalent to a **depth-first search** (DFS).

- The list is implemented as a **stack**
- The search goes deep into the tree, where the leaves often contain feasible solutions: the goal is to update the lower bound and the incumbent solution quickly
- The list stays short

FIFO

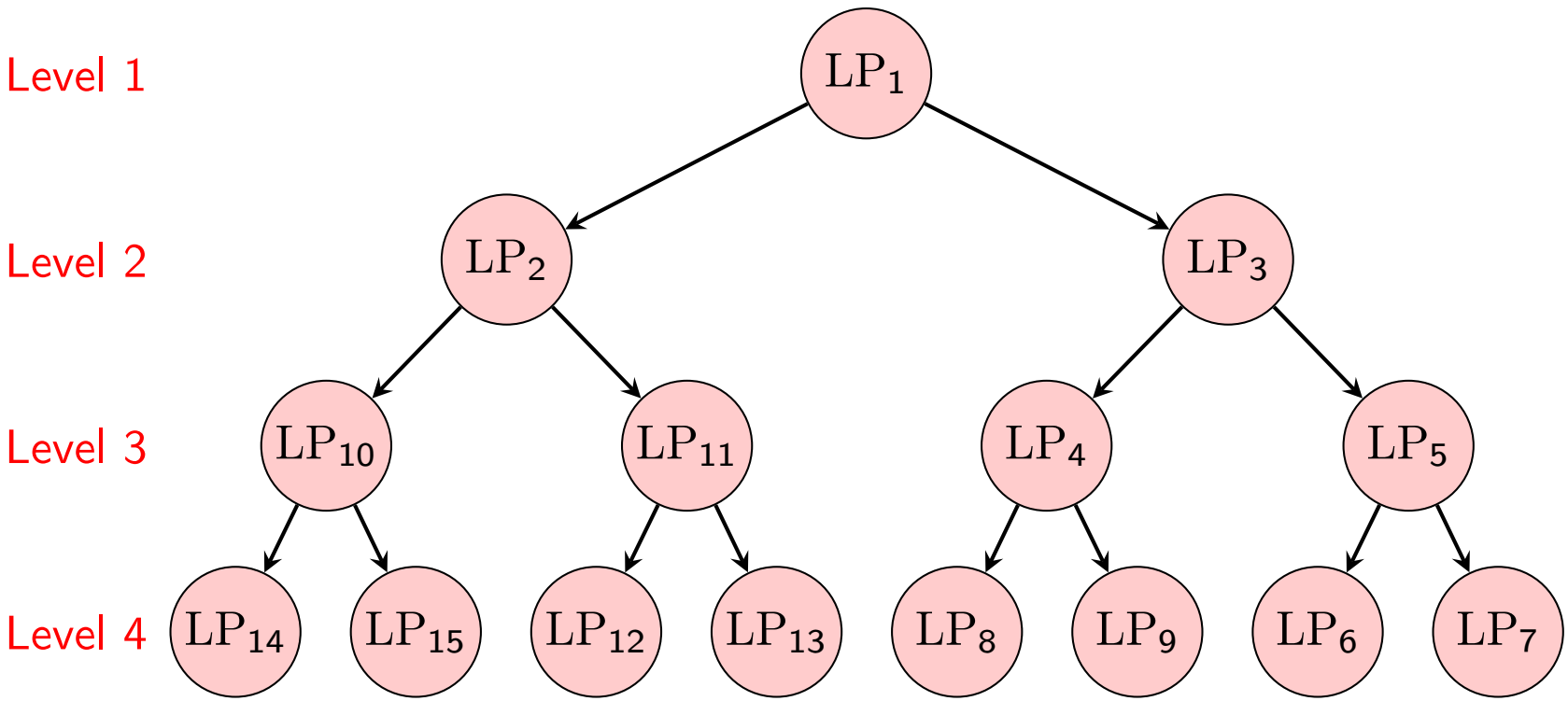
The **First-In-First-Out** (FIFO) strategy
processes the nodes in the order of insertion into the list L.

It is equivalent to a **breadth-first search** (BFS).

- The list is implemented as a **queue**
- The tree is explored level by level: this improves the upper bound, but usually it does not find good lower bounds quickly

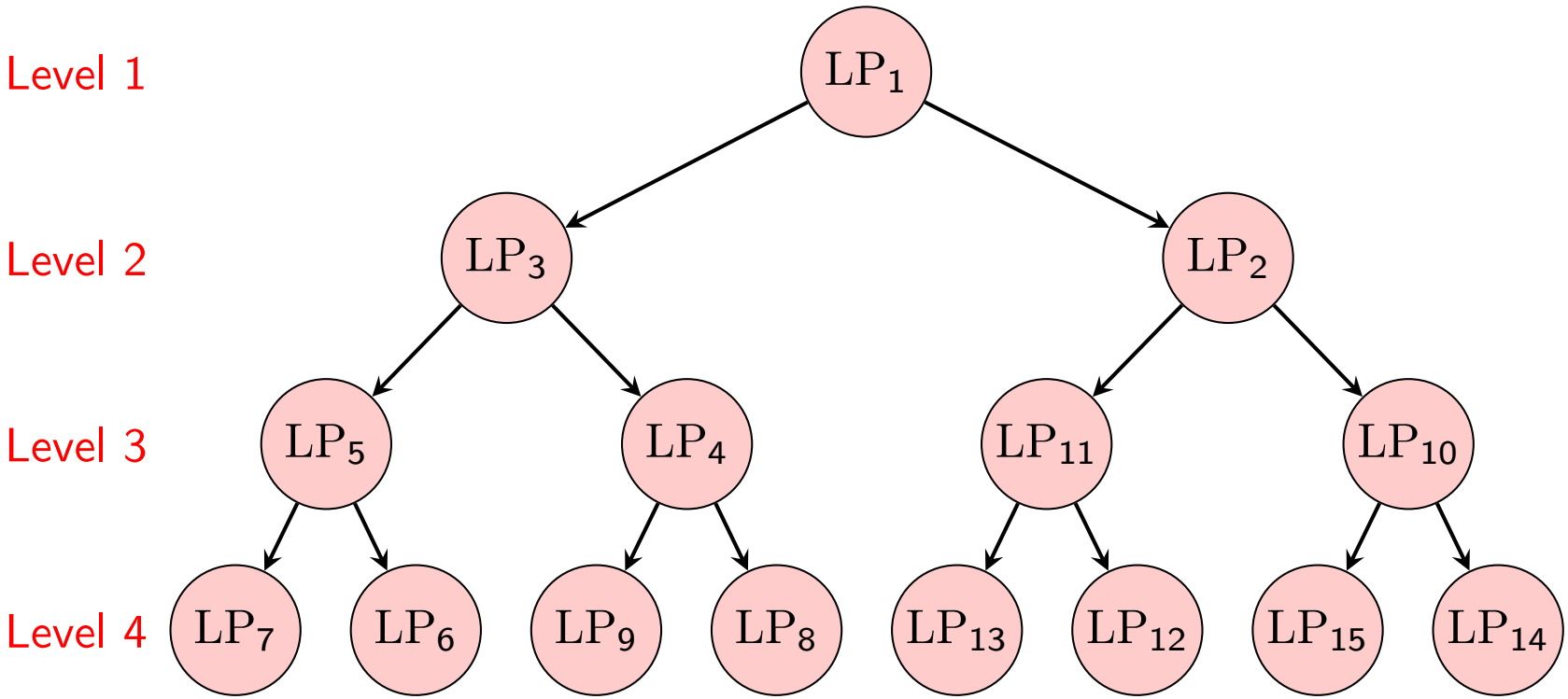
Example: LIFO strategy (1/2)

Order in which a complete binary branch-and-bound tree of 4 levels is explored with the LIFO strategy (no node is pruned). If the down-branching node is solved first, we have:



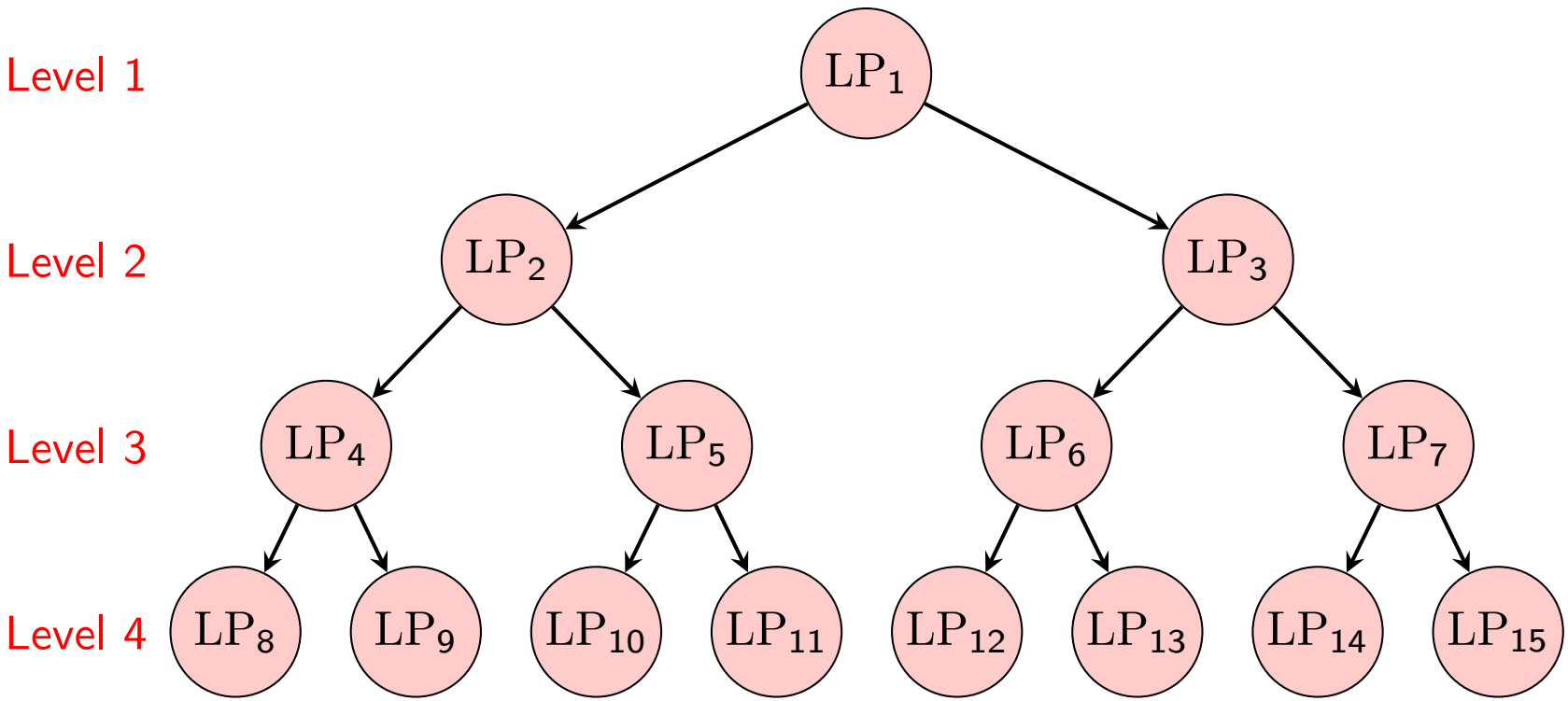
Example: LIFO strategy (2/2)

If the up-branching node is solved first, we have:



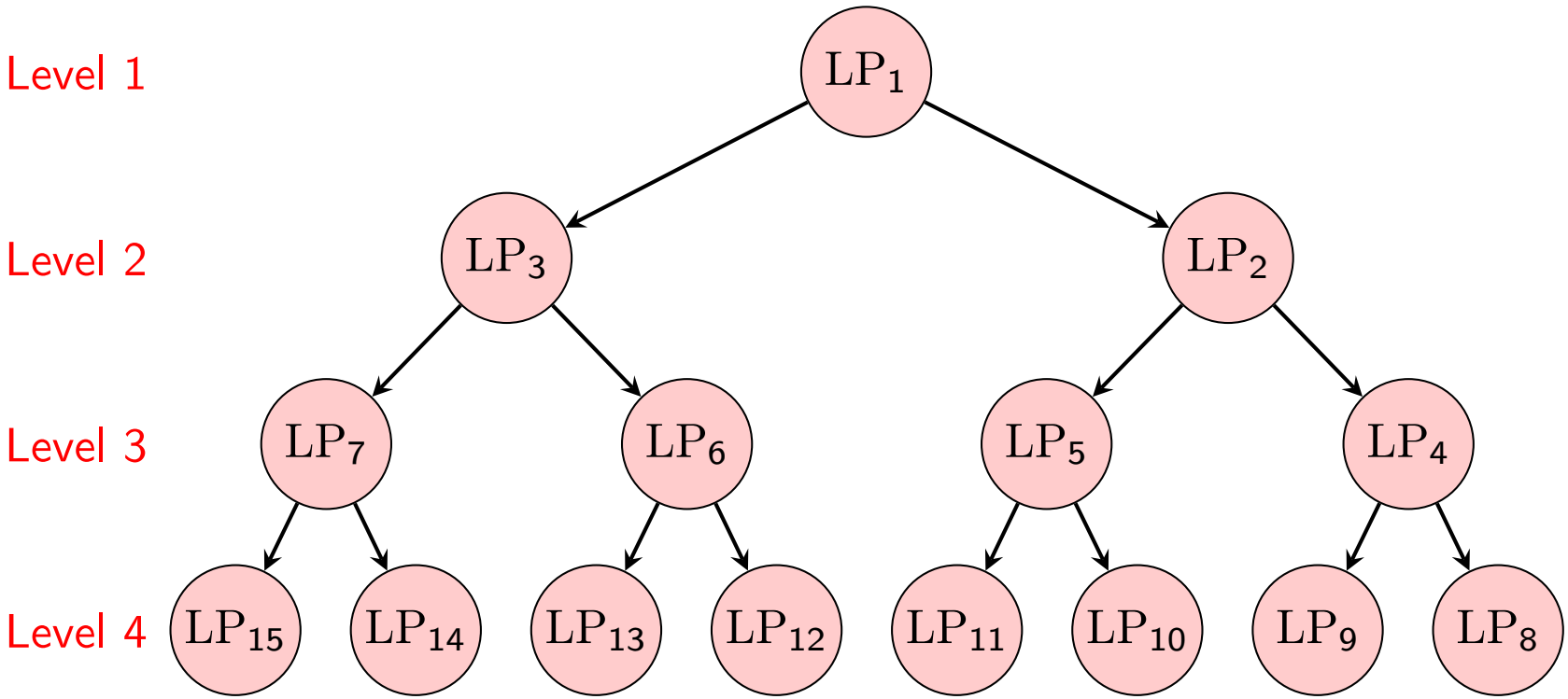
Example: FIFO strategy (1/2)

Order in which a complete binary branch-and-bound tree of 4 levels is explored with the FIFO strategy (no node is pruned). If the down-branching node is solved first, we have:



Example: FIFO strategy (2/2)

If the up-branching node is solved first, we have:



Summary

- **Best bound**: the node with the largest $z(\mathrm{LP}_k)$; the only way to decrease UB, few nodes but a long list
- **LIFO**: a stack, depth-first search; finds feasible solutions quickly, short list
- **FIFO**: a queue, breadth-first search; improves the upper bound, but finds good lower bounds slowly
- The order of the two children (down or up first) changes the numbering of the visit

Summary of the first part

- The B&B algorithm divides the feasible region with branching constraints on fractional integer variables, and it never loses a feasible solution
- Every node is pruned by infeasibility, by integrality or by bound, or it waits in the list L; the bound of a node is $\lfloor z(\mathrm{LP}_k) \rfloor$ when all the variables and costs are integer
- LB comes from the incumbent solution, UB from the nodes of the list; the skip rule avoids useless children
- Each child is re-optimized with the dual simplex algorithm from the dictionary of its parent
- In the second part: the complete executions of the algorithm on the car example